ECSE 4230: Embedded Systems

Morse Code

Group 7:
Tristan Besse
Jordan Howard
Nathan Park

11/14/2024

Roles and Responsibilities

- **Tristan Besse – Hardware**: Tristan wired the circuit for the project, including all of the sound and visual effects, while connecting the corresponding inputs to the Raspberry Pi. Tristial also wrote the section dedicated to the Audio Amplifier.
- **Jordan Howard—Documentation:** Jordan was responsible for writing and detailing the majority of the reporting for this project and testing the functionality of debounce. Jordan wrote the Algorithm and Debounce sections of the report and created the diagrams.
- **Nathan Park – Software:** Nathan was responsible for writing all of the software for the Morse code project. Nathan wrote the code for the encoder portion and the decoder portion of the project.

Contents

Deliverables

Generated Code

A link to the folder containing our code for part 1 and part 2 can be found below.
https://github.com/Herring-UGAECSE-4230-F24/group-7/tree/main/Python/MorseCode

Hardware Setup

Figure 1 shows our hardware setup for the project with the necessary components. **Figure 2** depicts the corresponding schematic for the hardware. It includes tags for the input and output of the Morse Keypad. The input from the keypad is derived from a voltage divider from the 5V rail of the Raspberry Pi. The voltage divider uses R3 and R4, both 910Ω resistors, to divide 5 volts to a 2.5V reference for the keypad input. This is necessary because a GPIO 27 is reading the output of the keypad, and the pin is not rated to handle 5V, but it can safely read a 2.5V signal as a HIGH.

The keypad output is connected to the LED circuit, created from a series resistor. The resistor limits the current flowing through the LED to prevent damage. The keypad output also holds the debounce circuit, which consists of a capacitor acting as a low-pass filter. The lefthand side of the schematic consists of the audio amplifier circuit meant to account for the speaker's adjustable noise for dots and dashes.

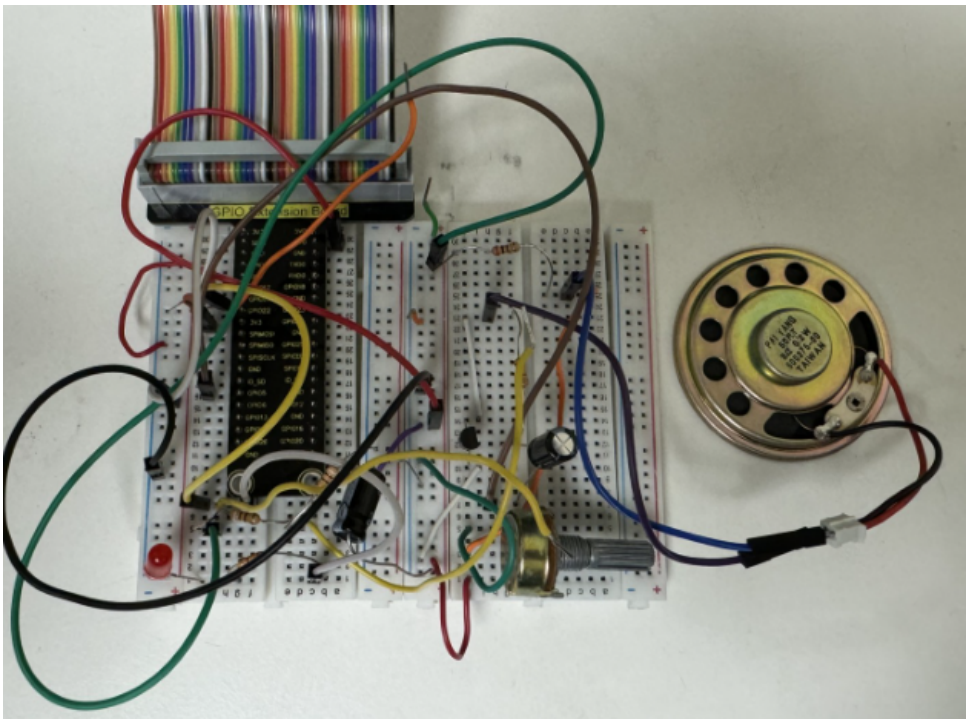


Figure 1 Electrical Hardware

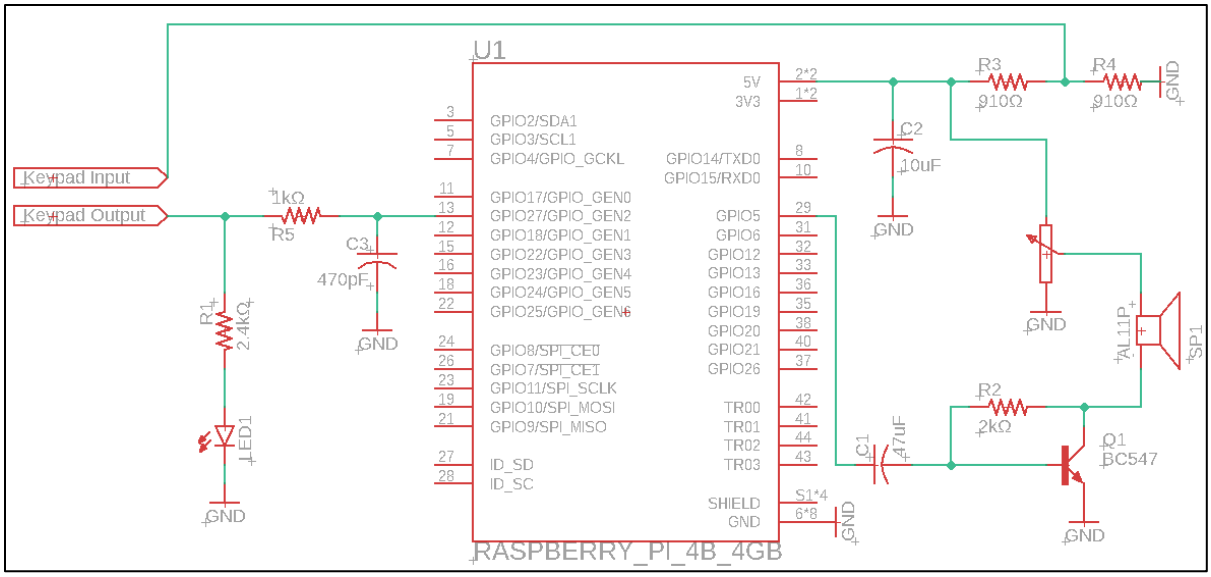


Figure 2 Electrical Schematic

Algorithm

Encoder

The encoder is meant to parse a text file and output a text file with the corresponding dots and dashes representing a Morse code message. It interacts with a speaker and LED to time the appropriate values for the dots and dashes. The pseudocode sequence is depicted in **Figure 3**. The program's most crucial aspect is assigning the proper dots and dashes for the correct letters and the correct spacing between the words. If the message is not encoded correctly, the wrong sequence plays for the LED and speakers.

The Python script starts by taking an example text file and creating an output text file for the encoded message, which consists of dots and dashes. The program reads each line of the file while the file still has lines denoted by the newline character ('\n'). For each line, the letters for each word will be matched to a dictionary containing all of the corresponding DOTs and DASHes for each letter. There will also be a SPACE between each character. Once a word is accounted for with the correct amount of dots and dashes, The spacing between each word will be accounted for inside each line as a WORD_SPACE. This process will repeat for each line inside the input file. Afterward, the list of encoded dots and dashes will be written in an output file with the corresponding word. When the output has finished being written, the program will close the input and output files.

The connection between the encoded list containing the appropriate DOTs, DASHes, SPACEs, and WORD_SPACEs can then sync with the speaker and LED. The program will allow the user to input a dot length in milliseconds. Once the average dot length is received, the program will begin to loop through each encoded value. The LED and speaker will turn on if the item is a dot or dash. Then, the program will sleep for a specific amount of time, depending on the value of the item. This process will continue throughout the list, and each value will also be printed on the terminal.

```
Procedure (mc_encoder.py)

CREATING OUTPUT FILE
(loop) while there are lines from input file
    (action) Read each line
    (loop) while there are words in the line
        (action) Match each letter in the word with its Morse Code value
        (action) Add a SPACE for each character
    (action) Add the WORD_SPACE to separate the words
(loop) while there are lines from encoded list
    print Morse Code values with the original word


CREATING SPEAKER AND LED EFFECT
(user input) Enter dot length in ms
(loop) while there are lines from encoded list
    (loop) for each item in the line
        (cond) if the item is a DOT or DASH
            (action) turn on speaker and LED
        (action) sleep for specified amount of time
```

Figure 3 Encoder Pseudocode

Decoder

The Decoder for the Morse Code project implements the physical Morse code keypad. It is a primitive device with basic functionality, but it is capable of relaying encoded messages through a telegraph key. Its operation involves connecting its input to a voltage while connecting the output to a GPIO pin that can then be used to read the signal from the telegraph key. An electrical signal will be created when the metal from the telegraph key is pressed against the meta pad.

The software consistently polls the telegraph GPIO pin. It will track the most recent moment when the key was pressed or released to determine DOTs, DASHes, CHAR_SPACEs, and WORD_SPACEs. The program uses a current and previous variable for the GPIO pin to track if a change was detected. If a press occurred, then the current status of the GPIO would be HIGH while the previous value would be LOW. This is due to the electrical signal being created when the telegraph key and the metal pad make contact, signifying a press. However, when a press is discerned, the program will begin calculating the space value on how long the telegraph key was released.

The logic behind determining presses and releases is reversed from the user's interaction with the system. This is due to the program tracking the time values, which can only be accounted for after an event occurs, as depicted in **Figure 4** with the rising and falling routine functions. When the user presses the telegraph key, the rising routine will be called. During this instance, the rise time is the most recent time because the user just pressed the keypad.

However, there is no distinct way of determining how long the key will be pressed from tracking that initial contact. This is required to determine the difference between a DASH and a DOT. This is why the logic is flipped from the user's interaction. Once the initial contact is made and the rising routine function can be called, the last fall time represents the last time the keypad was released. By subtracting the most recent press (last_rise_time) by the last release (last_fall_time), the function can then be used to determine if the prior release was a CHAR_SPACE or a WORD_SPACE because there are two known values from the physical interaction. The falling routine has a similar process to that of determining a DOT and a DASH.

```
86  # FUNCTIONS TO DETERMINE TIMING VALUES
87  def rising_routine():
88      global last_rise_time
89      last_rise_time = time() # current time
90      time_released = last_rise_time - last_fall_time  # duration the key is released
91      determine_release(time_released)
92      speaker.value = 0.5 # Turn speaker on due to the current press
93
94  def falling_routine():
95      global last_fall_time
96      last_fall_time = time() # current time
97      time_pressed = last_fall_time - last_rise_time  # duration the key is pressed
98      determine_press(time_pressed)
99      speaker.value = 0 # Turn speaker off due to the current release
00
```

Figure 4 Rising and Falling Routines

Once the different Morse code values can be distinguished, they can be matched to the corresponding letters and words. To determine the timing of a DOT, the attention calibration must be completed when the program first starts. Based on these interactions, a sequence of [DASH, DOT, DASH, DOT, DASH] will be used to determine the average dot length. Following the calibration, the user can start decoding messages and each letter and word will be printed to the terminal. When the user is done typing, they must key out the special phrase "OUT" in the following sequence [DOT, DASH, DOT, DASH, DOT]. Finally, a text file will be created with their corresponding message. **Figure 5** shows a comprehensive flowchart of the decoding algorithm.

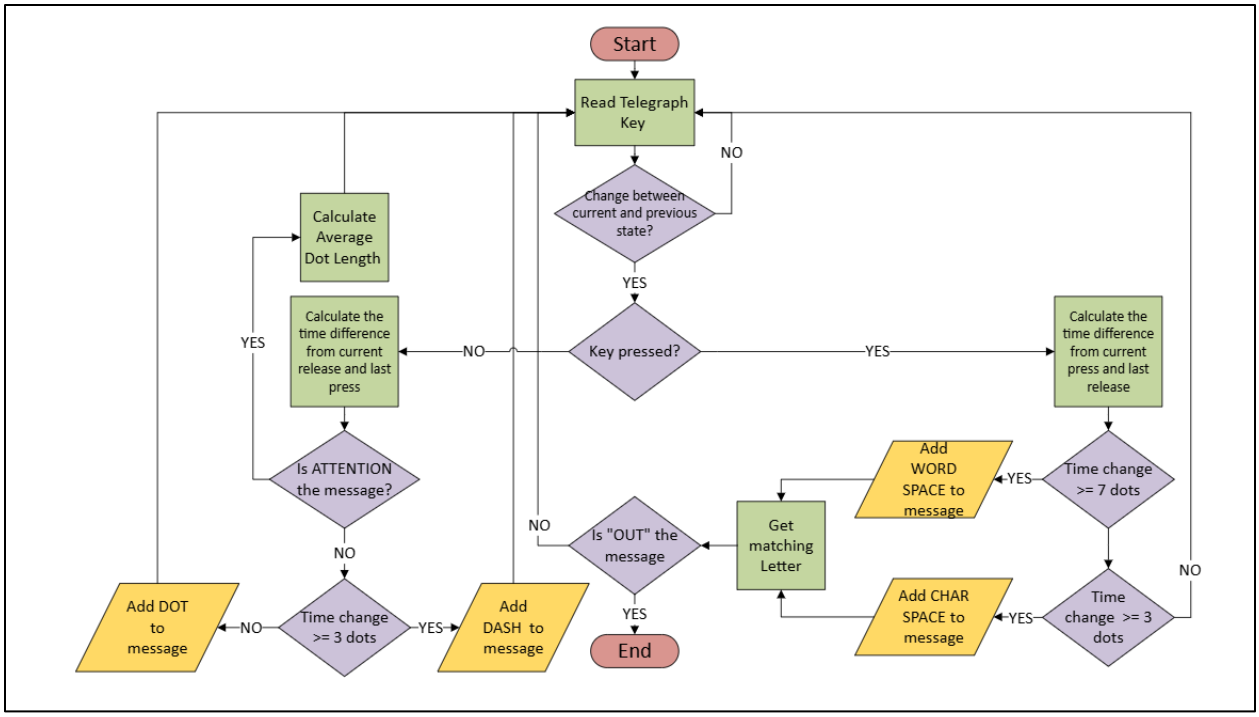


Figure 5 Decoder Flowchart

Debounce

Software

The leading software debounce was from the Decoder program's Is Rising and Is Falling functions, shown in **Figure 6**. The program will continuously poll the GPIO pin of the telegraph key from **Figure 7**. The Is Rising function will check if the current GPIO value is HIGH for the telegraph and the previous value was LOW. This will account for situations when the telegraph key has just been pressed since the input is now HIGH, which means before it was LOW and can only account for that specific instance of changing from LOW to HIGH and reduce debounce situations. A similar process occurs for the Is Falling function. It is also important to note that a sleep of 1 millisecond is added at the end of each polling loop to provide additional support in solving debounce issues due to the Python lines being able to be interpreted so quickly, which may lead to duplicate readings.

Without comparing to the previous state, the function would be unable to distinguish between a new press and a continued press, leading to potential misinterpretation of signals. The function would simply return True whenever the current value is HIGH, regardless of how long it has been in that state. This would cause the decoder program to do a series of duplicate values with this debounce method.

For example, if you press and hold the telegraph key, the current value will remain HIGH for multiple polling cycles. Without checking the previous value, every cycle would be interpreted as a new press, leading to multiple detections of what should be a single press. That is why having a previous value at the end of the polling sequence is necessary to provide a reference when checking to see if there was a change in the state of the GPIO based on user interaction.

```
def is_rising(current, prev):
    return (current and not prev)

def is_falling(current, prev):
    return (not current and prev)
```

Figure 6 Is Rising and Falling Functions

```
while True: # Continuos Polling
    current_val = GPIO.input(telegraph_pin)
    if is_rising(current_val, prev_val): rising_routine()
    if is_falling(current_val, prev_val): falling_routine()
    prev_val = current_val
    sleep(0.01) # Delay
```

Figure 7 Polling Loop

Hardware

The hardware option for the debounce consisted of a small capacitor to smooth the signal from the telegraph key. The configuration is shown in **Figure 8**, and the circuit is a passive low-pass filter. The capacitor value is small after testing several capacitance values for the circuit. We noticed that the telegraph keypad did not have major issues with debounce when using a control test to measure the response of the presses with an o-scope, as shown in **Figure 9**. That is why we used a relatively small capacitor to handle small spikes when a press was detected to help attenuate high-frequency signals.

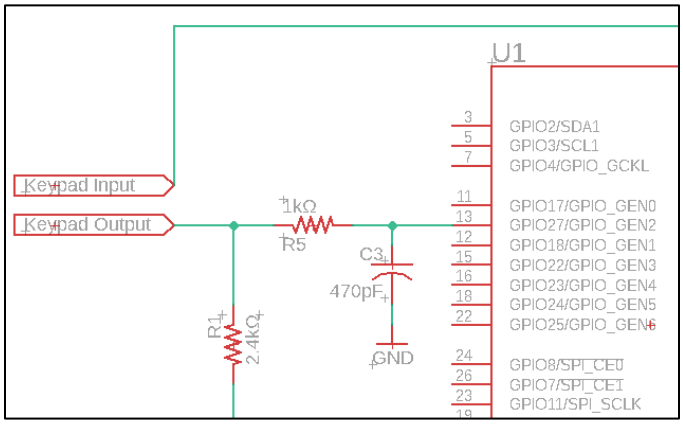


Figure 8 Hardware Debounce Circuit

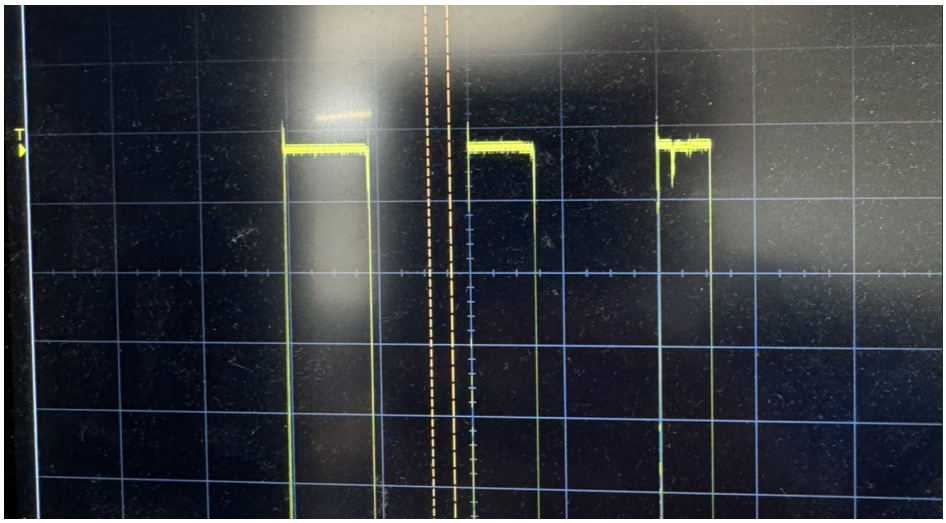


Figure 9 Control Test with No Hardware or Software Debounce

Comparison

While both options could have been applied, our software implementation provided statistically better performance than the hardware option. This was mainly because the program needed to account for presses and releases. The software debounces accounts for releases and presses, while the hardware option only accounts for presses. The project also does not

require precise timing within microseconds, so a 1-millisecond delay while polling will not make a noticeable difference in the accuracy of the Morse code keypad.

To test the two implementations quantifiably, we conducted a series of trials where we would do 5 Dot presses on the keypad. We removed the capacitor C3 and resistor R5 in the software test from **Figure 8**. For the hardware implementation, we did not account for the previous variable when polling the GPIO pin in the Is Rising Function and removed the sleep function from the polling loop. **Figure 9** and **Figure 10** shows the results from the Software-only and Hardware-only implementation for the output to the terminal. As expected, the Software-only test properly accounted for all 5 presses. However, the Hardware-only test was unable to account for the spacing, which is why there are multiple word spaces in the hardware implementation that do not account for the releases. This also explains why the terminal reads multiple E values since the letter E represents one Dot.

Figure 11 and **Figure 12** show the waveforms for the Software-only and Hardware-only tests. The Software-only test is considerably smoother because when the user first interacts with the GPIO pin, only that initial signal will be accounted for until the release of the GPIO pin. The sleep function of 1 millisecond also helped ignore false values. The Hardware-only implementation was able to smooth out most of the noise from the original control test. However, the characteristics of the filter will still cause small ripples within the signal.

Debouncing issues can be detrimental to any system where a button or switch is pressed, like this project's telegraph key. When pressed or released, the key does not immediately settle into its final state of HIGH or LOW. Instead, it may bounce between states for a few fractions of a second, causing multiple transitions to be detected. This bouncing can lead to correct readings if appropriately handled. The debounce option our group ultimately decided on was the software-based option due to its simplicity in implementation and the added benefit of the telegraph key, which also accounts for the spacing with the software.

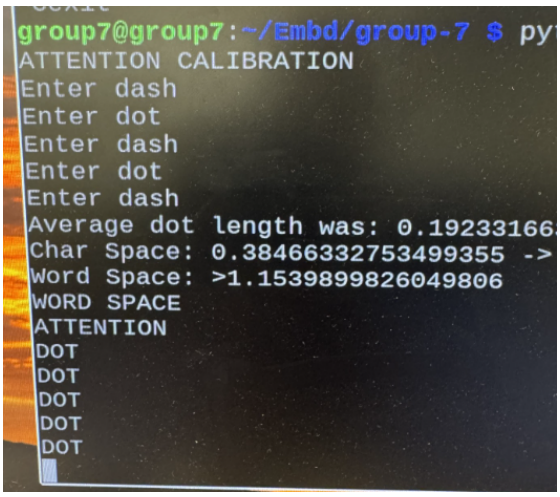


Figure 9 Software-Only Terminal Output

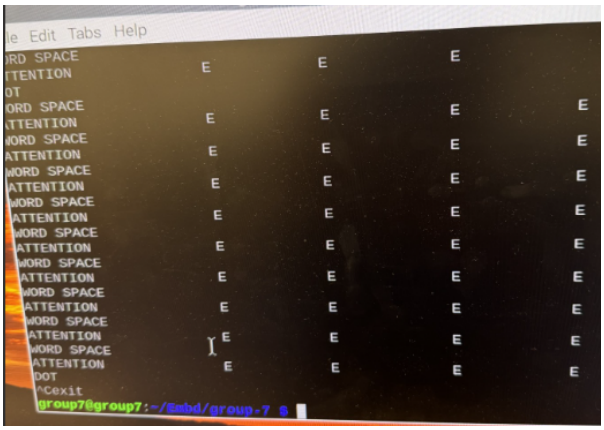


Figure 10 Hardware-Only Terminal Output

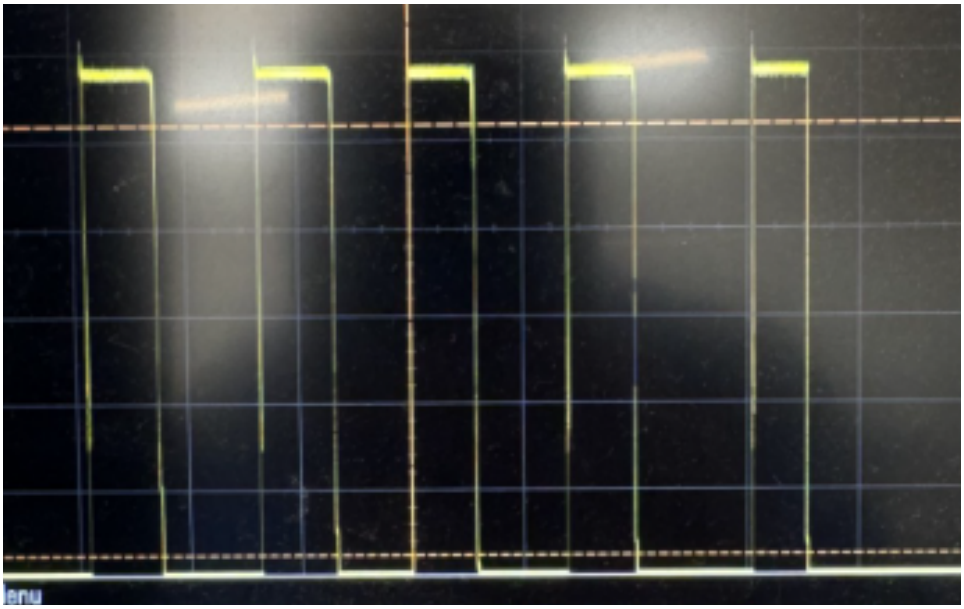


Figure 11 Software-Only O-scope Reading

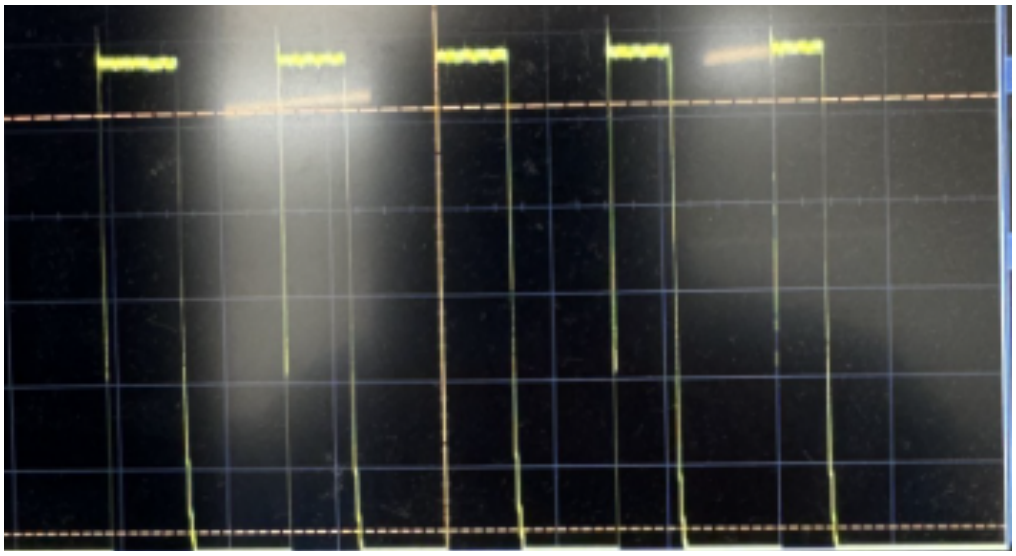


Figure 12 Hardware-Only O-scope Reading

Audio Amplifier

An audio amplifier circuit (depicted in **Figure 13**) was modified to amplify a 3.3V 500Hz audio signal that goes across the 8-ohm speaker through BJT current biasing and a VCC of 5VDC.

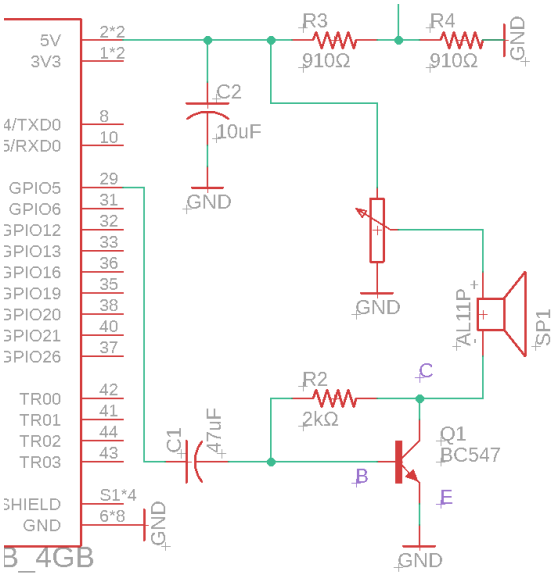


Figure 13 Audio Amplifier Circuit

The essential design of the circuit was to bias the bipolar junction transistor, BC547, by changing the voltage suddenly at the base of the transistor while the transistor was in common emitter configuration. The expected relationship of the voltage of V_{BE} being around 660 to 700 mV per datasheet value shown in **Appendix 1**. The three modes of the transistor is shown at different times during this simulation, those being: active (when circuit was at stable expect V_{BE}), saturation (when V_{Base} was increased by 3.3V), and cutoff (when the V_{Base} was decreased by 3.3V). DC analysis of the circuit was done below along with simulation evidence for what is occurring during the operation of the amplifier. It should be noted that the capacitor prevents DC bias from passing through to the base of the transistor.

Saturation:

At t=1 in the simulation [**Figure 15**], the base is set to 3.3V via the PWM input, meaning that node B is 3.3V plus the V_{BE} or around 3.9-4.0V at that instant that the voltage is elevated. At this point the transistor is in saturation, but the feedback of the 2k-ohm resistor is going to bring the voltage back to the stable level of V_{BE} (as well as limiting the current going into the base of the transistor).This led to the voltage level at V_{CE} or V_C to increase to around 2.6V [**Figure 15 @ t=1s**]. The current of I_b is flowing from base to collector and was represented by ohms law of $(V_{CC}-V_C)/R_{speaker}$ or 300mA. The current of I_C is flowing from V_{CC} to collector and can be represented through ohms law of $(V_B-V_C)/R_{feedback}$, or approximately 650uA which is then limited by the resistor and allows the voltage at V_B to drop back to around 0.66V and return to active mode.

Active (Stable):

After the feedback resets the voltage at V_{BE} or V_B to around 620mV [**Figure 15 @ 1s<t<1.45s**] and stays in this stable region until the PWM signal changes state. In active mode the V_C is to be around 660mV (per simulation) which is nearing the max value of the datasheet value of 600mV. For simplicity, the datasheet voltage was used to calculate the current running from collector to base. This calculation was done similarly to the calculations done in Saturation by means of ohms law. I_B was found to be 25uA. I_C flows from V_{cc} to the collector. I_C through $R_{speaker}$ was 0.5438A.

Cutoff:

Once the voltage switches states again, from 3.3V to 0V, there is an instantaneous change of -3.3V at V_B. Considering the switching is happening from active/stable state the V_{BE} of 620mV is now affected and changed [**Figure 15 at t=1.5s**] to be 0.62V-3.3V resulting in a V_B value of -2.68V. From simulation [**Figure 15**] V_{CE} is found to be around 2.58V. Utilizing these values the current can now be calculated using ohms law. I_B is now running collector to base and is found to be 302.5mA. I_C flows from V_{cc} to the collector. I_C is 302.5mA.

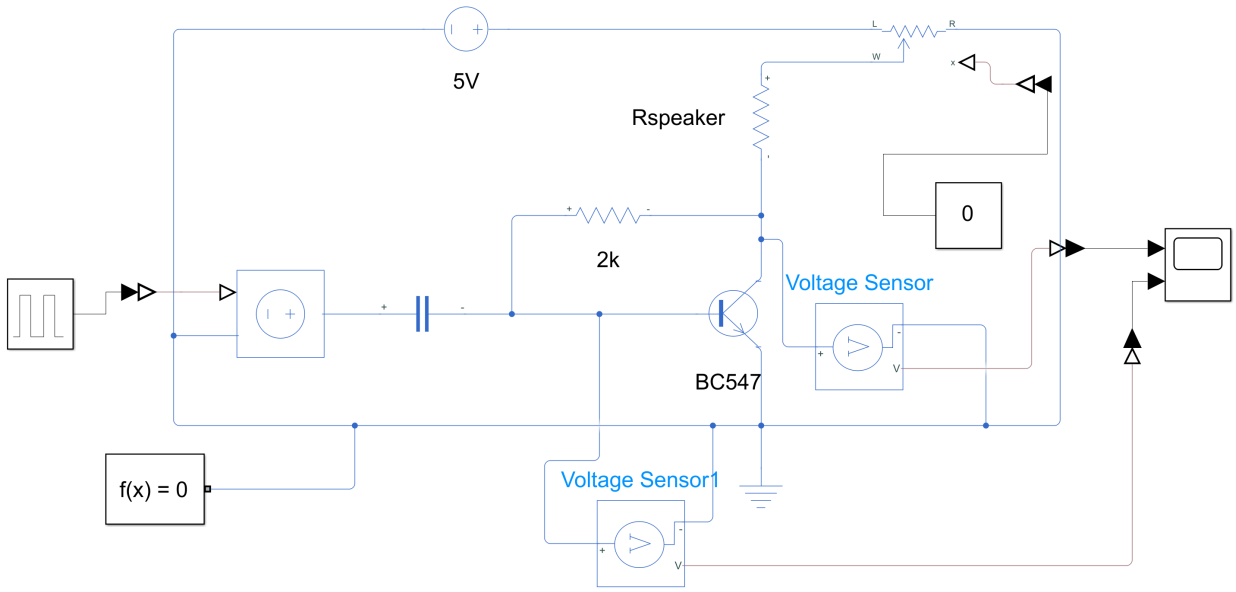


Figure 14 Audio Amplifier Simulink

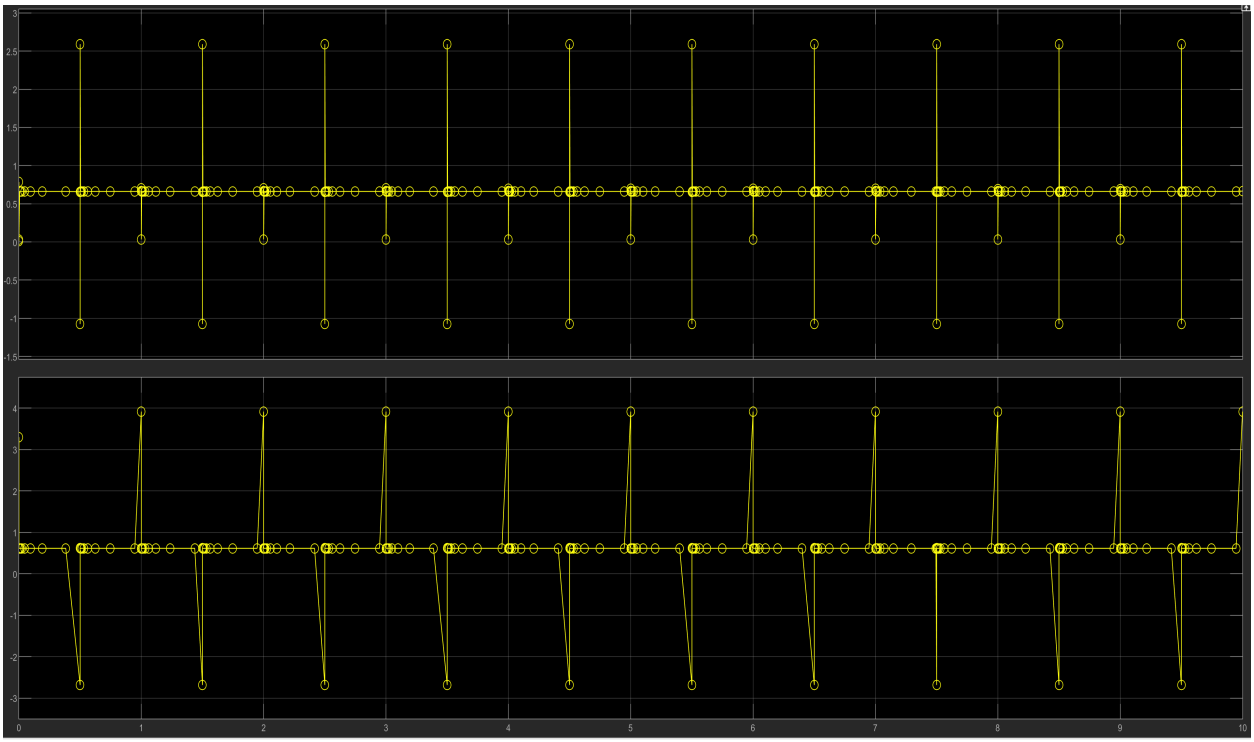


Figure 15 Above Graph Represents VCE versus Time | Below Graph Represents VBE versus Time

Potentiometer Voltage Control:

Additionally, the final circuit includes a potentiometer wired 5V (V_{CC}) to pin 3, ground to pin 1, and the wiper of the potentiometer to pin 2. The potentiometer was implemented to be a voltage divider with the speaker as shown in **Figure 14** for volume control. When the resistance of the potentiometer increases, the voltage across the potentiometer drops, also dropping the total voltage difference across the speaker. With less voltage being supplied to the speaker there is also less power consumed by the speaker (per P=I*V) resulting in a decreased audible noise of the speaker.

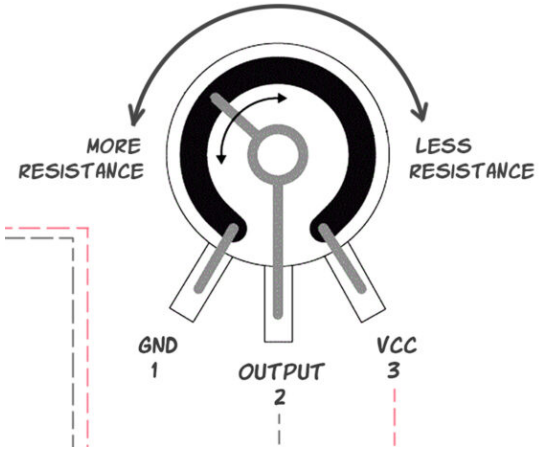


Figure 16 Potentiometer Representation

References

1. "time — Time access and conversions." Python 3.14.0 Documentation, Python Software Foundation, docs.python.org/3/library/time.html.
2. Pankaj. "How To Use the Python time sleep() Method." DigitalOcean, www.digitalocean.com/community/tutorials/python-time-sleep.
3. Santos, Sara, and Rui Santos. "Raspberry Pi: PWM Outputs with Python (Fading LED)." Random Nerd Tutorials, randomnerdtutorials.com/raspberry-pi-pwm-python/.
4. "Inputs." *Raspberry-gpio-python*, SourceForge, https://sourceforge.net/p/raspberry-gpio-python/wiki/Inputs/.
5. Dr. Yao, “ELEE 3270 Lecture 5: BJT,” University of Georgia, Athens, GA, USA, Lecture Notes, Oct. 2023.

Appendix

a1. BC547 Datasheet:

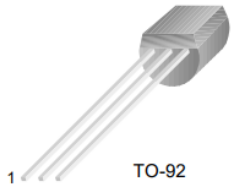
FAIRCHILD SEMICONDUCTOR®

BC546/547/548/549/550

BC546/547/548/549/550

Switching and Applications

- High Voltage: BC546, V_{CEO}=65V
- Low Noise: BC549, BC550
- Complement to BC556 ... BC560



TO-92

1. Collector 2. Base 3. Emitter

NPN Epitaxial Silicon Transistor

Absolute Maximum Ratings T_a=25°C unless otherwise noted

Symbol	Parameter	Value	Units
V_{CBO}	Collector-Base Voltage : BC546	80	V
	: BC547/550	50	V
	: BC548/549	30	V
V_{CEO}	Collector-Emitter Voltage : BC546	65	V
	: BC547/550	45	V
	: BC548/549	30	V
V_{EBO}	Emitter-Base Voltage : BC546/547	6	V
	: BC548/549/550	5	V
I_C	Collector Current (DC)	100	mA
P_C	Collector Power Dissipation	500	mW
T_J	Junction Temperature	150	°C
T_{STG}	Storage Temperature	-65 ~ 150	°C

Electrical Characteristics T_a=25°C unless otherwise noted

Symbol	Parameter	Test Condition	Min.	Typ.	Max.	Units
I_{CBO}	Collector Cut-off Current	V_{CB}=30V, I_E=0			15	nA
h_{FE}	DC Current Gain	V_{CE}=5V, I_C=2mA	110		800	
V_{CE} (sat)	Collector-Emitter Saturation Voltage	I_C=10mA, I_B=0.5mA		90	250	mV
		I_C=100mA, I_B=5mA		200	600	mV
V_{BE} (sat)	Base-Emitter Saturation Voltage	I_C=10mA, I_B=0.5mA		700		mV
		I_C=100mA, I_B=5mA		900		mV
V_{BE} (on)	Base-Emitter On Voltage	V_{CE}=5V, I_C=2mA	580	660	700	mV
		V_{CE}=5V, I_C=10mA			720	mV
f_T	Current Gain Bandwidth Product	V_{CE}=5V, I_C=10mA, f=100MHz		300		MHz
C_{ob}	Output Capacitance	V_{CB}=10V, I_E=0, f=1MHz		3.5	6	pF
C_{ib}	Input Capacitance	V_{EB}=0.5V, I_C=0, f=1MHz		9		pF
NF	Noise Figure : BC546/547/548	V_{CE}=5V, I_C=200µA		2	10	dB
	: BC549/550	f=1KHz, R_G=2KΩ		1.2	4	dB
	: BC549	V_{CE}=5V, I_C=200µA		1.4	4	dB
	: BC550	R_G=2KΩ, f=30~15000MHz		1.4	3	dB

h_{FE} Classification

Classification	A	B	C
h_{FE}	110 ~ 220	200 ~ 450	420 ~ 800

Rev. A2, August 2002

Typical Characteristics

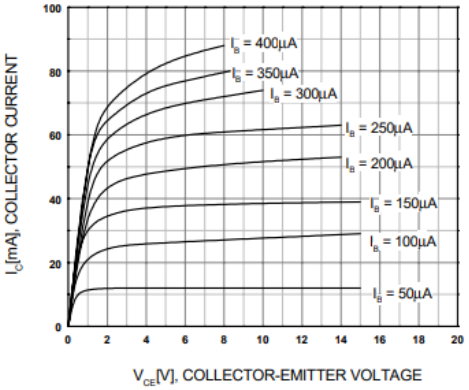


Figure 1. Static Characteristic

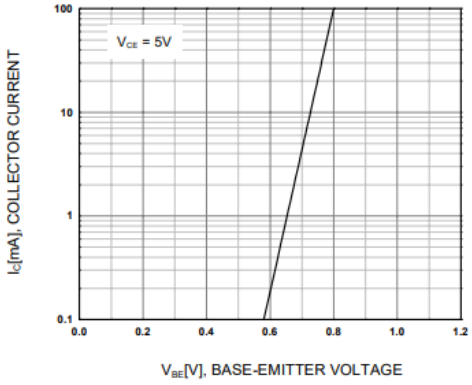


Figure 2. Transfer Characteristic

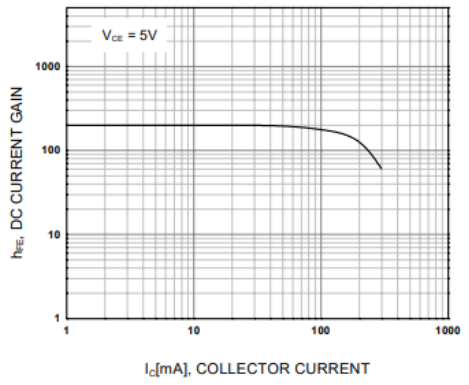


Figure 3. DC current Gain

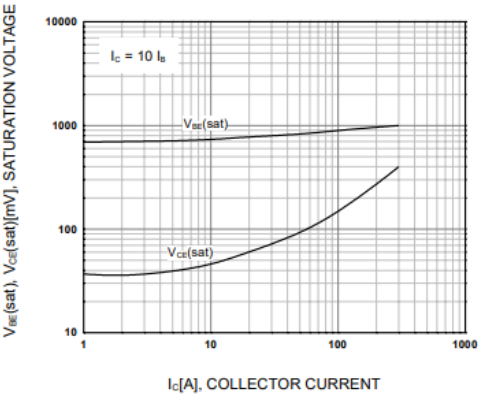


Figure 4. Base-Emitter Saturation Voltage Collector-Emitter Saturation Voltage

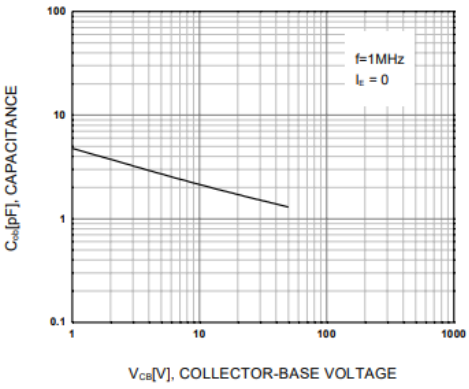


Figure 5. Output Capacitance

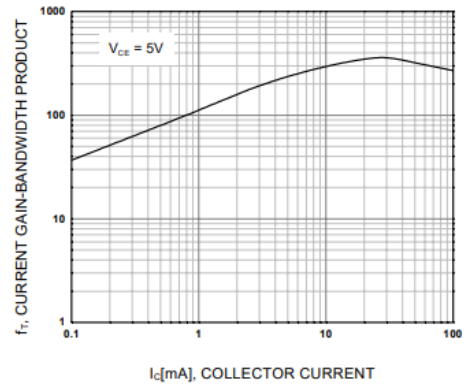
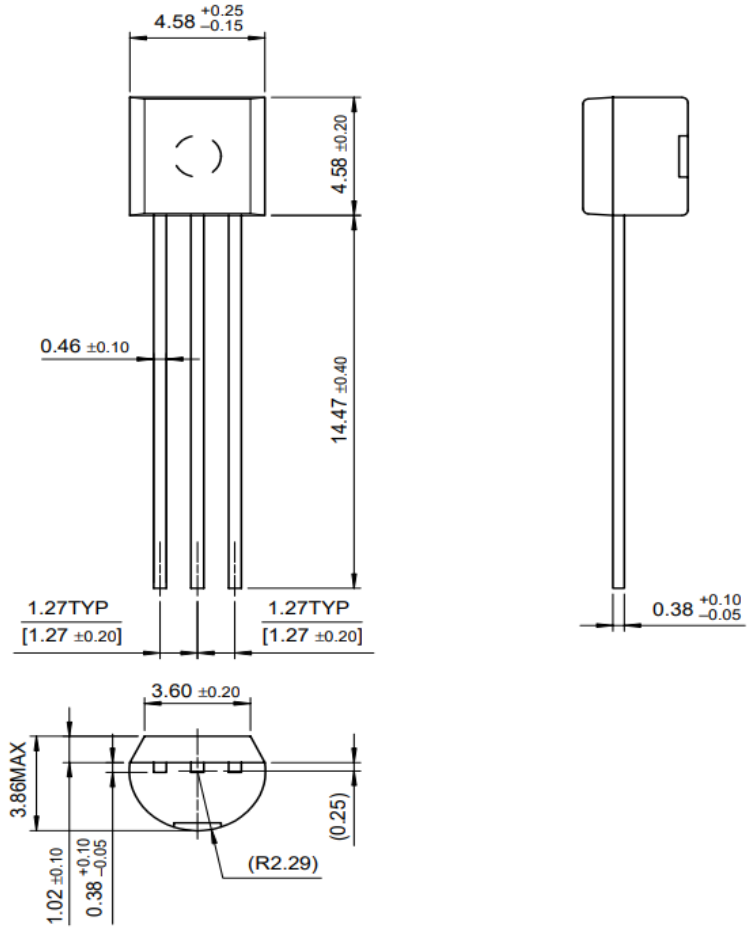


Figure 6. Current Gain Bandwidth Product

Package Dimensions

TO-92



Dimensions in Millimeters

Rev. A2, August 2002

TRADEMARKS

The following are registered and unregistered trademarks Fairchild Semiconductor owns or is authorized to use and is not intended to be an exhaustive list of all such trademarks.

ACEx™	FACT™	ImpliedDisconnect™	PACMAN™	SPM™
ActiveArray™	FACT Quiet series™	ISOPLANAR™	POP™	Stealth™
Bottomless™	FAST®	LittleFET™	Power247™	SuperSOT™-3
CoolFET™	FASTr™	MicroFET™	PowerTrench®	SuperSOT™-6
CROSSVOLT™	FRFET™	MicroPak™	QFET™	SuperSOT™-8
DOME™	GlobalOptoisolator™	MICROWIRE™	QS™	SyncFET™
EcoSPARK™	GTO™	MSX™	QT Optoelectronics™	TinyLogic™
E^2CMOS™	HiSeC™	MSXPro™	Quiet Series™	TruTranslation™
EnSigna™	I^2C™	OCX™	RapidConfigure™	UHC™
Across the board. Around the world.™		OCXPro™	RapidConnect™	UltraFET®
The Power Franchise™		OPTOLOGIC®	SILENT SWITCHER®	VCX™
Programmable Active Droop™		OPTOPLANAR™	SMART START™	

DISCLAIMER

FAIRCHILD SEMICONDUCTOR RESERVES THE RIGHT TO MAKE CHANGES WITHOUT FURTHER NOTICE TO ANY PRODUCTS HEREIN TO IMPROVE RELIABILITY, FUNCTION OR DESIGN. FAIRCHILD DOES NOT ASSUME ANY LIABILITY ARISING OUT OF THE APPLICATION OR USE OF ANY PRODUCT OR CIRCUIT DESCRIBED HEREIN; NEITHER DOES IT CONVEY ANY LICENSE UNDER ITS PATENT RIGHTS, NOR THE RIGHTS OF OTHERS.

LIFE SUPPORT POLICY

FAIRCHILD'S PRODUCTS ARE NOT AUTHORIZED FOR USE AS CRITICAL COMPONENTS IN LIFE SUPPORT DEVICES OR SYSTEMS WITHOUT THE EXPRESS WRITTEN APPROVAL OF FAIRCHILD SEMICONDUCTOR CORPORATION.

As used herein:

1. Life support devices or systems are devices or systems which, (a) are intended for surgical implant into the body, or (b) support or sustain life, or (c) whose failure to perform when properly used in accordance with instructions for use provided in the labeling, can be reasonably expected to result in significant injury to the user.

2. A critical component is any component of a life support device or system whose failure to perform can be reasonably expected to cause the failure of the life support device or system, or to affect its safety or effectiveness.

PRODUCT STATUS DEFINITIONS

Definition of Terms

Datasheet Identification	Product Status	Definition
Advance Information	Formative or In Design	This datasheet contains the design specifications for product development. Specifications may change in any manner without notice.
Preliminary	First Production	This datasheet contains preliminary data, and supplementary data will be published at a later date. Fairchild Semiconductor reserves the right to make changes at any time without notice in order to improve design.
No Identification Needed	Full Production	This datasheet contains final specifications. Fairchild Semiconductor reserves the right to make changes at any time without notice in order to improve design.
Obsolete	Not In Production	This datasheet contains specifications on a product that has been discontinued by Fairchild semiconductor. The datasheet is printed for reference information only.